



The bridge to possible

# Cloud-Native Architecture Advancements

## The Evolution of the Catalyst Switch in the Meraki Cloud

Nico Darrow – Technical Leader, Product Management

Tony Carmichael – Sr Director, Product Management

BRKARC-1975

CISCO *Live!*

#CiscoLive

# Cisco Webex App

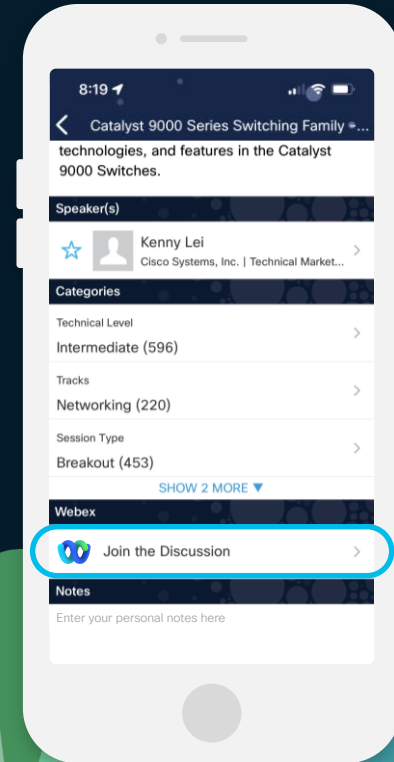
## Questions?

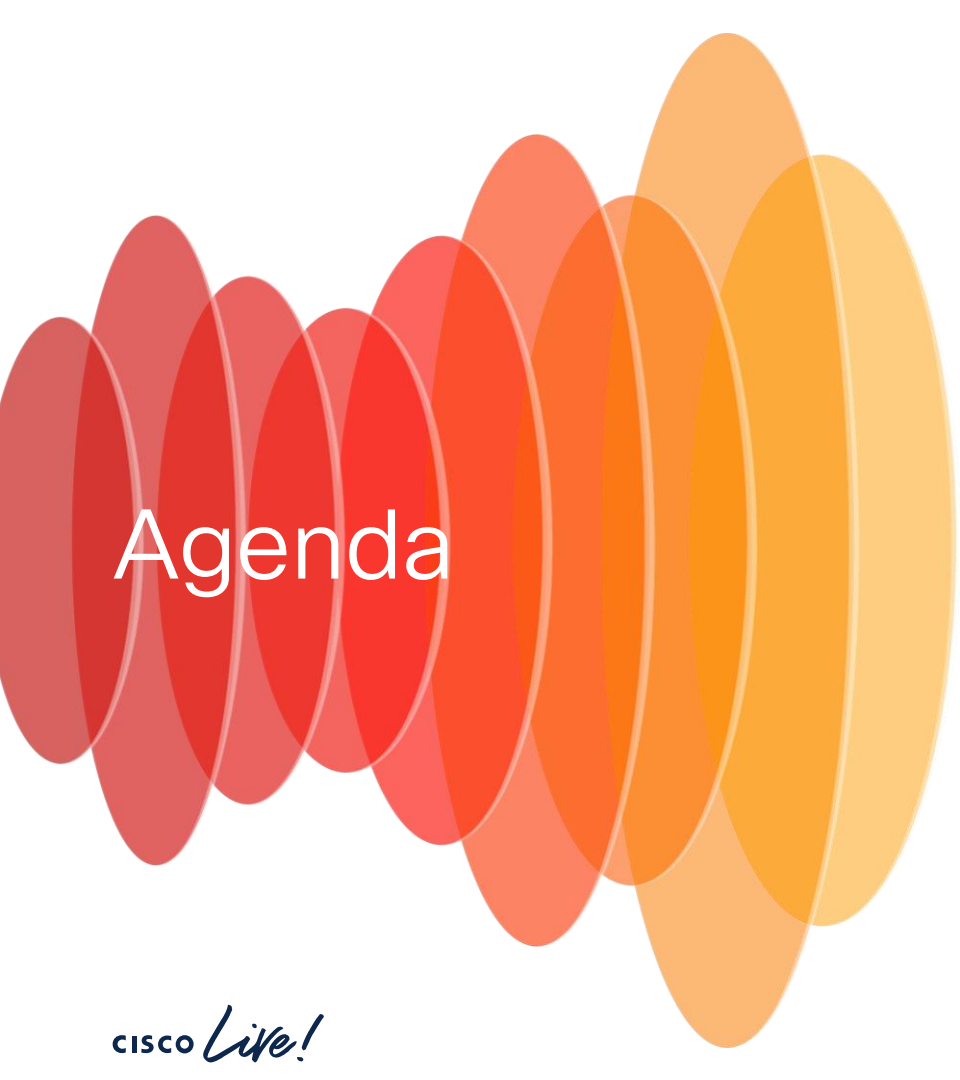
Use Cisco Webex App to chat with the speaker after the session

## How

- 1 Find this session in the Cisco Live Mobile App
- 2 Click “Join the Discussion”
- 3 Install the Webex App or go directly to the Webex space
- 4 Enter messages/questions in the Webex space

Webex spaces will be moderated by the speaker until June 7, 2024.

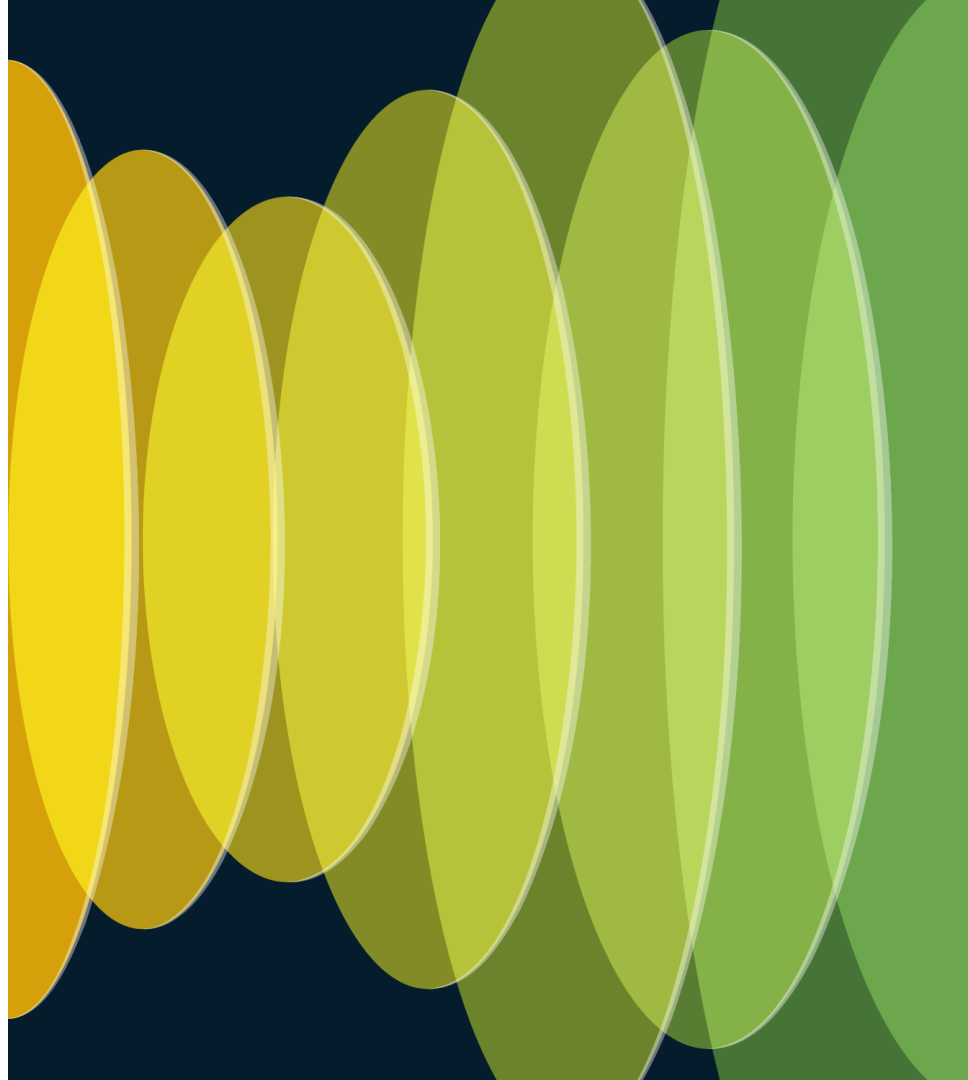




# Agenda

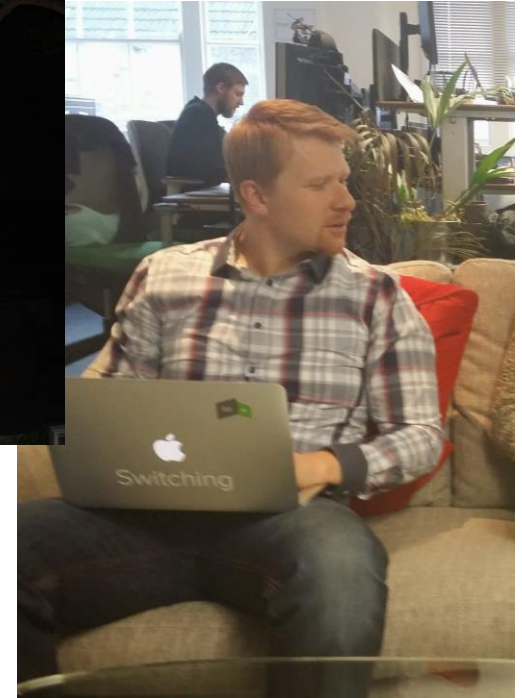
- Introduction
- Storytime
- Cloud Born Hardware
- The Origin Story
- Architectural Evolution
- Optimizations
- Conclusion

# Storytime



# Tony Tony Tony

- First time unboxing a c3650
- 12 years later
  - What is the product experience?
  - Changing the experience in a 2024 modern way

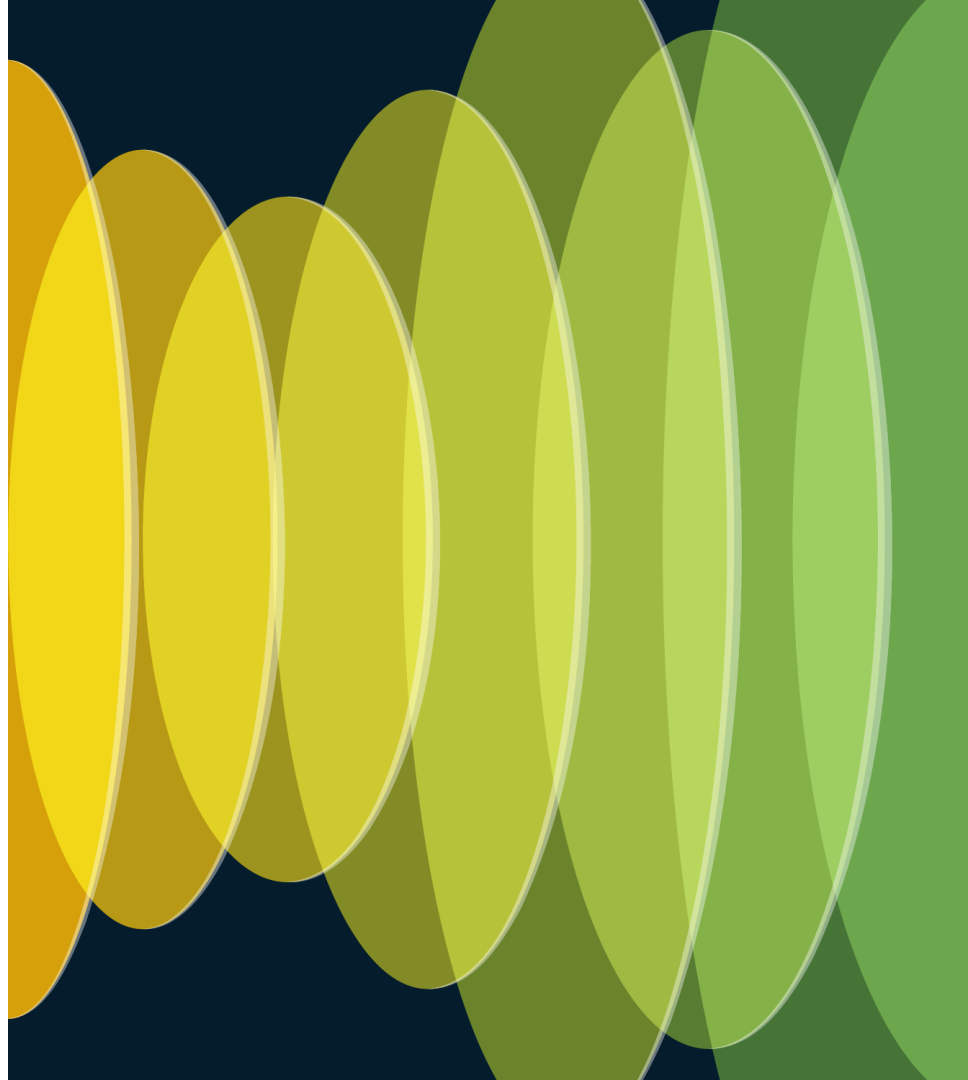


# Nico Darrow

- Maker/Hacker/Geek
  - Robotics Lover
  - Maker and Chronic Tinkerer
  - Python Geek
- 14 year Cisco veteran
  - Enterprise/Premier Architect
- Life-long Meraki Believer
  - Programmability became the new superpower



# Cloud Born Hardware



# MS390: the first Catalyst based cloud-managed switch



Based off the first generation of 9300s packaged in a cloud-native out of box runtime

Brought new capabilities to the cloud for the first time. Power stacking, ADP, AVC w/ deep packet inspection

# Zero-Touch provisioning out of the box

*No change to customer experience and ease of deployment*

Frictionless, web-based configuration and onboarding



No pre-staging  
No onsite configuration



Unbox and plug in devices  
at their final destinations



Smart LED indicators  
help guide installer cabling

Configure from anywhere, deploy everywhere.



Speed up deployment  
and maximize uptime



Reduce deployment costs  
and travel needs



Minimize potential errors  
and standardize settings

# A Family of 24 Catalyst 9300/X/L-M Options!

## Gigabit

Up to 1 GbE



### C9300L 24/48T-4X

- 24/48x 1G/100M copper
- Data only
- 4x SFP+ 1/10G fixed fiber uplink



### C9300L 24/48P-4X

- 24/48x 1G/100M copper
- 30W PoE
- 4x SFP+ 1/10G fixed fiber uplink



### C9300L 48PF-4X

- 48x 1G/100M copper
- 30W PoE
- 4x SFP+ 1/10G fixed fiber uplink



### C9300 24/48T

- 24/48x 1G copper
- Data only
- Modular uplink options



### C9300 24/48P

- 24/48x 1G copper
- 30W PoE+
- Modular uplink options



### C9300 24/48U

- 24/48x 1G copper
- 60W UPOE
- Modular uplink options

## MultiGig

Up to 10 GbE (mGig)



### C9300L 24/48UXG-4X

- 24/48x 10G mGig
- 60W PoE
- 4x SFP+ 1/10G fixed fiber uplink



### C9300X 48TX

- 48x 10G mGig
- Data only
- Modular uplink options



### C9300X 24/48HX

- 24/48x 10G mGig
- 90W UPOE+
- Modular uplink options



### C9300X 48HXN

- 40x 5G and 8x 10G mGig
- 90W UPOE+
- Modular uplink options



### C9300 24UX

- 24x 10G mGig
- 60W UPOE
- Modular uplink options



### C9300 48UXM

- 36x 2.5G mGig and 12x 10G mGig
- 60W UPOE
- Modular uplink options



### C9300 48UN

- 48x 5G mGig
- 60W UPOE
- Modular uplink options

## Fiber aggregation

10/1 GbE SFP+ fiber



### C9300 24S/48S

- 24/48 port 1G SFP
- 40G/10G/1G fiber modular uplink options



### C9300X 12/24Y

- 25G/10G/1G (SFP28)
- 40G/10G/1G fiber modular uplink options

### Catalyst 9300X modular uplinks\*



2x QSFP28 40/100G



8x SFP28 1/10/25G

Note: 9300X modules are not compatible with 9300

### StackWise® and StackPower cables



50CM-1M-3M 50 cm-1 m-3 m 30 cm-150 cm

### Platinum-rated power supplies



350W AC 715W AC/DC 1100W AC 1900W AC

### Catalyst 9300 modular uplinks\*



4x 10G/1G SFP



8x 10G/1G SFP/SFP+

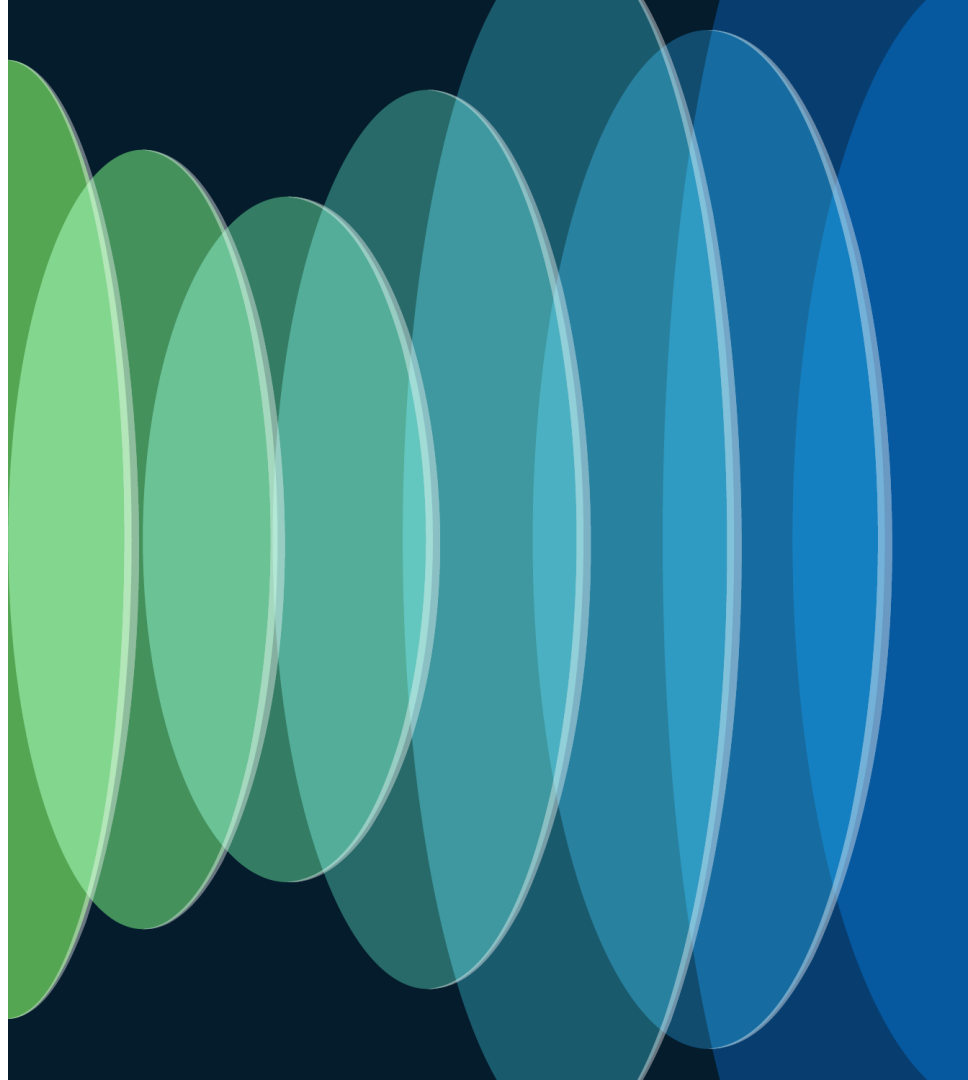


2x 40G QSFP



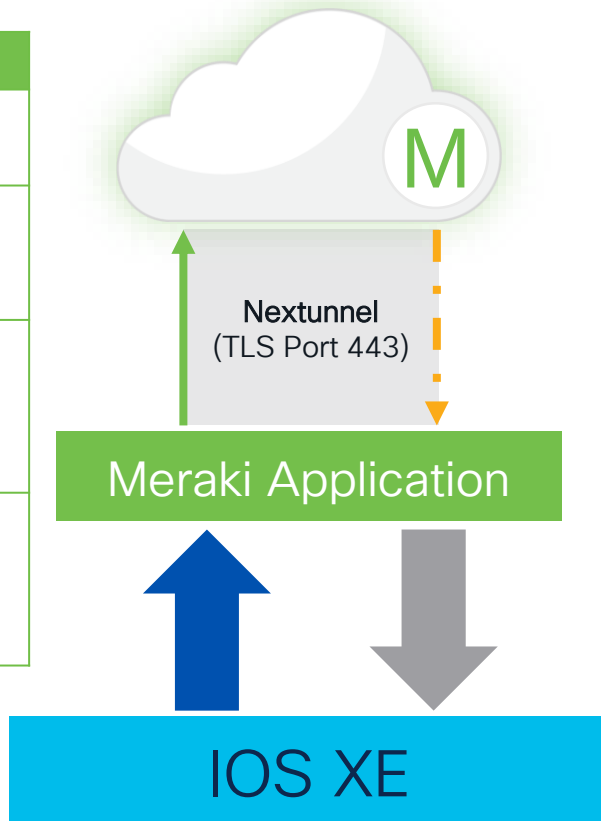
2x 25G/10G/1G SFP/SFP+

# The Origin Story



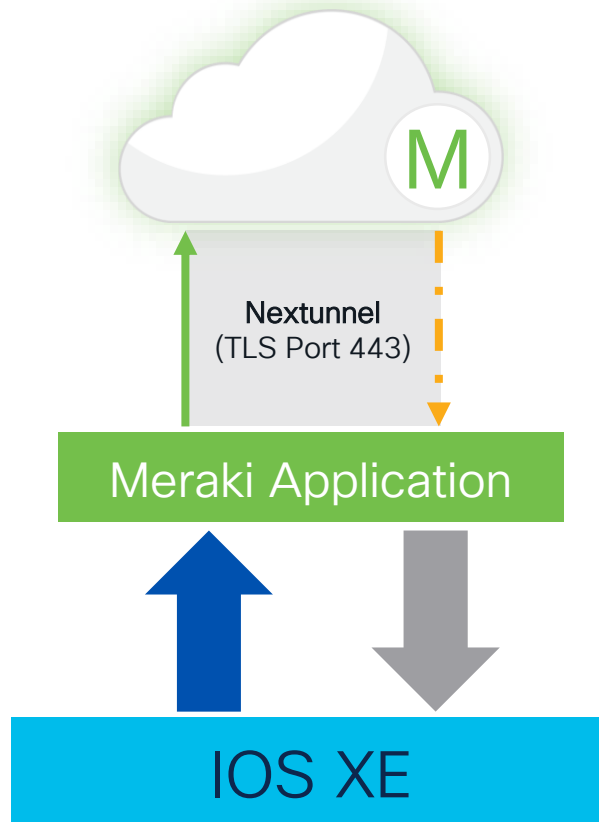
# 1<sup>st</sup> Gen Software Architecture – App to Cloud

| Function              | Use                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------|
| Heartbeat             | Checks in to dashboard and evaluates state flags                                           |
| Packet Capture Stream | Stream packet capture to dashboard browser proxy                                           |
| Config File Download  | Pulls the configuration file and translates config lines to XML yang modeled configuration |
| Firmware Proxy        | Downloads firmware and stages for either an app update or IOS XE + app update              |



| Function            | Use                                        |
|---------------------|--------------------------------------------|
| Protobuf Grabber    | Captures telemetry and stores in dashboard |
| Live Tool Execution | Proxy live tool command to Meraki App      |

# 1<sup>st</sup> Gen Software Architecture - App to XE



| Function                   | Use                                              |
|----------------------------|--------------------------------------------------|
| NetConf Oper Subscriptions | Telemetry capture and store as protobuf          |
| Packet Capture             | Stream packet capture to dashboard browser proxy |
| IPFIX Stream               | Client and application visibility and tracking   |

| Function                 | Use                                      |
|--------------------------|------------------------------------------|
| NetConf Write Operations | Configuration Deploy & update            |
| Live Tool Execution      | Run Tools executed by admin in dashboard |
| Firmware Management      | Deploys firmware installation to IOS XE  |

# Challenges and Learnings



Platform dependent  
development

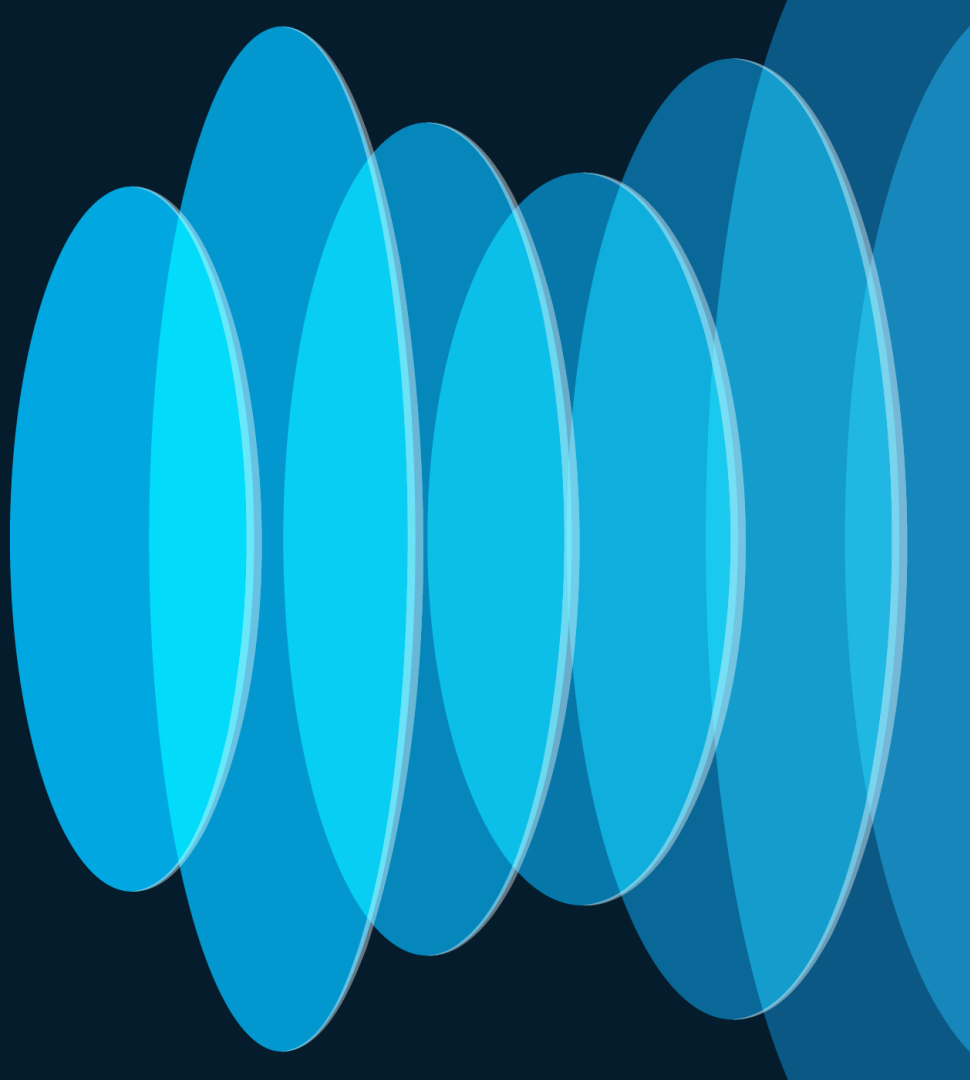
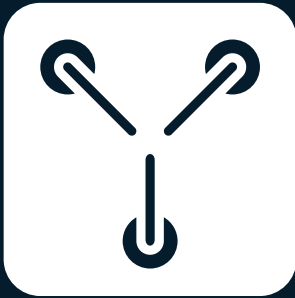


Limited agility in  
software architecture



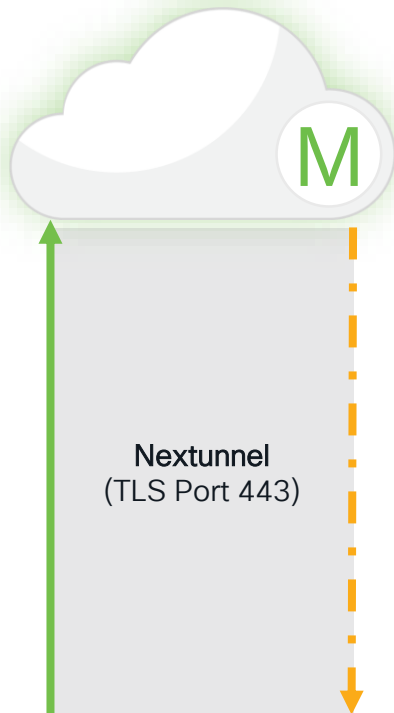
Multiple abstraction layers  
and software stacks

*Where we're  
going, we don't  
need containers*



# How did we improve the architecture?

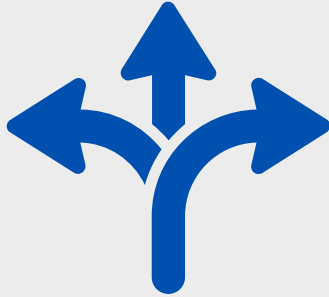
| Function                         | Use                                                                                                           |
|----------------------------------|---------------------------------------------------------------------------------------------------------------|
| Uplink Auto Configuration (UAC)  | Dynamic interface creation using discovery mechanisms maintaining connectivity to dashboard                   |
| Connect Service                  | Checks in to dashboard and evaluates state flags                                                              |
| Packet Capture                   | Stream packet capture to dashboard browser proxy                                                              |
| Config File DL/UL and deployment | Uploads current configuration and pulls XML formatted configuration to deploy locally against NetConf Service |
| Local Status Page                | Hosted in IOS XE instead of in app, providing local configuration tools                                       |
| Telemetry Cache                  | Stores telemetry on box for Dashboard grabbers to retrieve                                                    |



| Function               | Use                                                                |
|------------------------|--------------------------------------------------------------------|
| Telemetry Grabbers     | Pulls telemetry directly from IOS XE for presentation in Dashboard |
| Live Tool RPC          | Executes Live Tools via NetConf RPC                                |
| Packet Capture Trigger | Executes packet capture process using NetConf RPC                  |



# Resilient connectivity to the Cloud



## Uplink Auto Config (UAC)

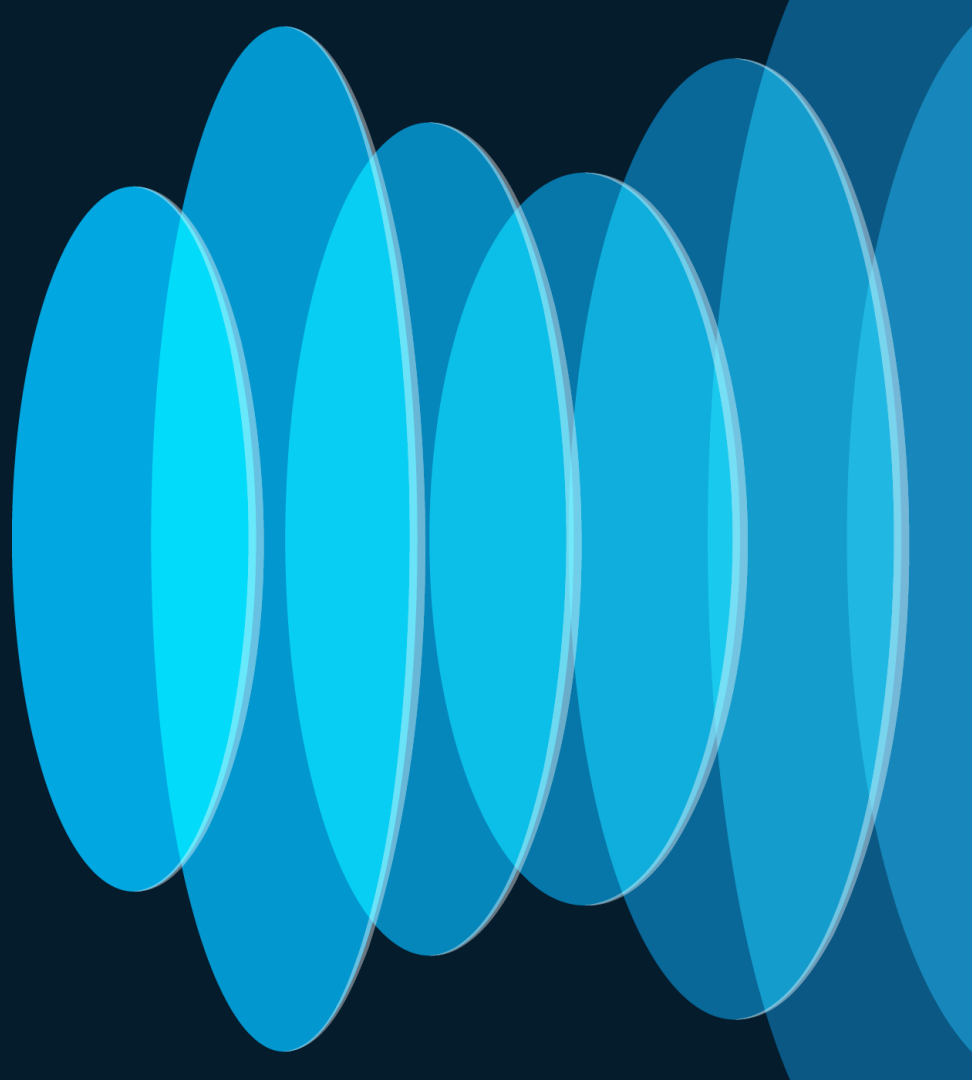
- Automated discovery for dashboard connectivity
- Automated failover / path discovery
- Tunable for primary interface
- Creates stack-ranked interface list



## Safe Config Rollback

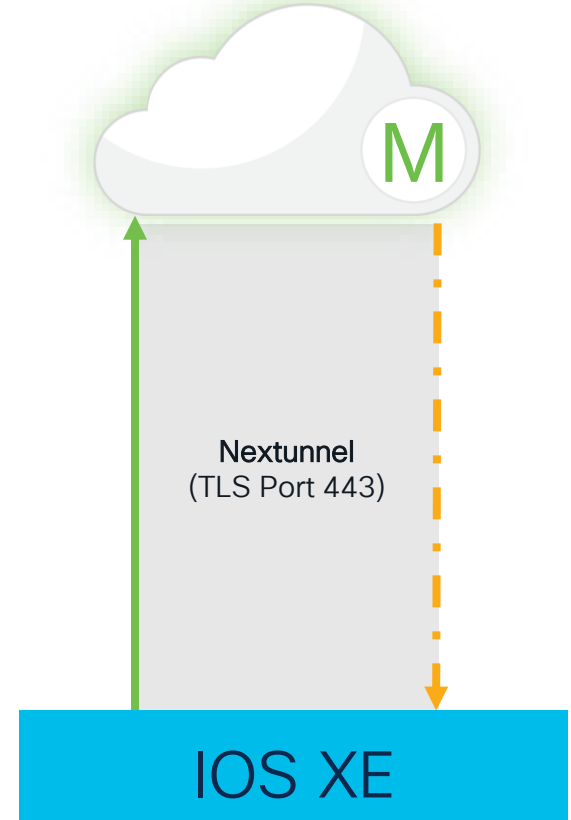
- Recovery mechanism for configuration error
- Marks configuration safe if dashboard connectivity is maintained after change
- Will roll-back to safe config if dashboard access is lost after configuration change

# Architectural Evolution



# What is changing with 2.0?

Meraki Methods  
with  
Catalyst Interfaces



# Configurations built natively for IOS XE

- Dashboard can talk YANG now!
- Developed directly in dashboard
  - Validation of complete configs before pushing them
  - Leverage existing Meraki tooling
- Feature development independent of firmware upgrades
- Rapid resolution in the event of a configuration-based bug

```
<TenGigabitEthernet>
  <name>1/0/20</name>
  <ip>
    <dhcp>
      <snooping xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-dhcp">
        <trust/>
      </snooping>
    </dhcp>
    <flow xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-flow">
      <monitor-new>
        <name>MERAKI_AVC_IPV4</name>
        <direction>input</direction>
      </monitor-new>
      <monitor-new>
        <name>MERAKI_AVC_IPV4</name>
        <direction>output</direction>
      </monitor-new>
      <monitor-new>
        <name>meraki_monitor</name>
        <direction>input</direction>
      </monitor-new>
    </flow>
  </ip>
</TenGigabitEthernet>
```

# A simple example

The screenshot shows the Cisco Meraki 'Update 1 port' configuration window. The left sidebar contains the following menu items: Network (Kendall Klub), Network-wide, Security & SD-WAN, Switching, Wireless, Cameras, Sensors, Organization, Admin, and Find in Menu. The main configuration area is titled 'Update 1 port' and contains the following fields and options:

- Switch / Port: The Switch / 5
- Name:
- Port status:
- Auto negotiate:
- Port schedule:
- Tags:
- Type:
- Native VLAN:
- Allowed VLANs:
- RSTP:
- STP guard:
- Port isolation:
- Trusted DAI:

At the bottom right of the window are 'Cancel' and 'Update' buttons. A large blue arrow points from the 'Switching' menu item in the left sidebar to the 'Port status' field in the configuration form.

# A simple example

Meraki Config State

```
port5:enabled true
```

# A simple example

## Meraki Config State

```
port5:enabled true
```

## Meraki Diff

```
- port5:enabled false  
+ port5:enabled true
```

# A simple example

## Meraki Config State

```
port5:enabled true
```

## Meraki Diff

```
- port5:enabled false  
+ port5:enabled true
```

## IOSXE Diff

```
Device(config)# interface gigabitethernet1/0/5  
Device(config-if)# no shutdown  
Device(config-if)# end
```

# A simple example

## Meraki Config State

```
port5:enabled true
```

## Meraki Diff

```
- port5:enabled false  
+ port5:enabled true
```

## IOSXE Diff

```
Device(config)# interface gigabitethernet1/0/5  
Device(config-if)# no shutdown  
Device(config-if)# end
```

## NETCONF Diff



```
<config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0">  
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">  
    <interface>  
      <GigabitEthernet>  
        <name>1/0/5</name>  
        <shutdown nc:operation="remove" />  
      </GigabitEthernet>  
    </interface>  
  </native>  
</config>
```

# A simple example

## NETCONF Diff

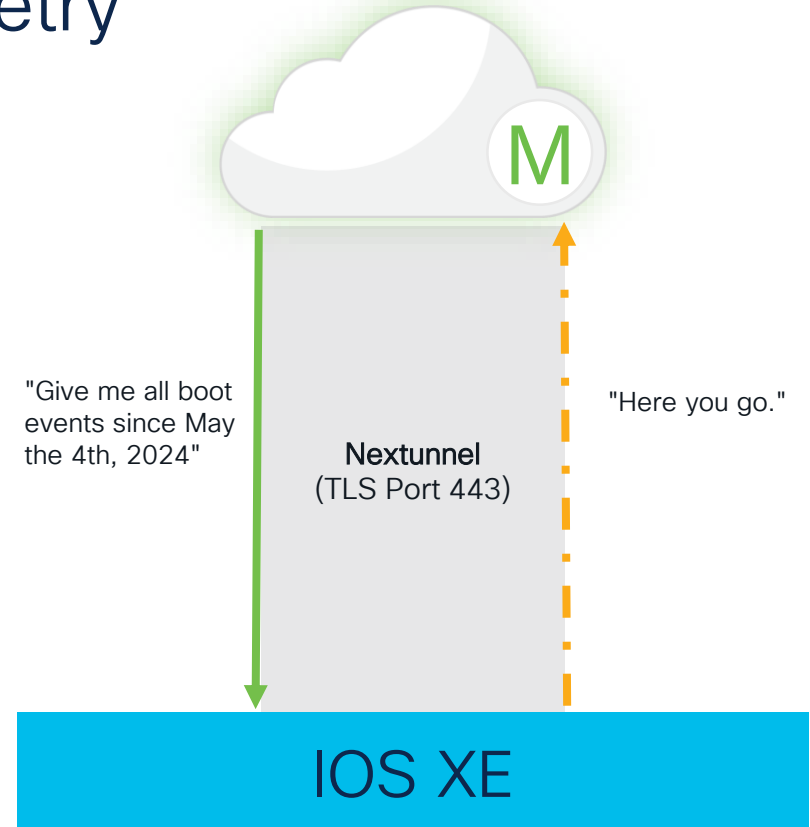
```
<config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0">
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
    <interface>
      <GigabitEthernet>
        <name>1/0/5</name>
        <shutdown nc:operation="remove" />
      </GigabitEthernet>
    </interface>
  </native>
</config>
```

## XPath Diff

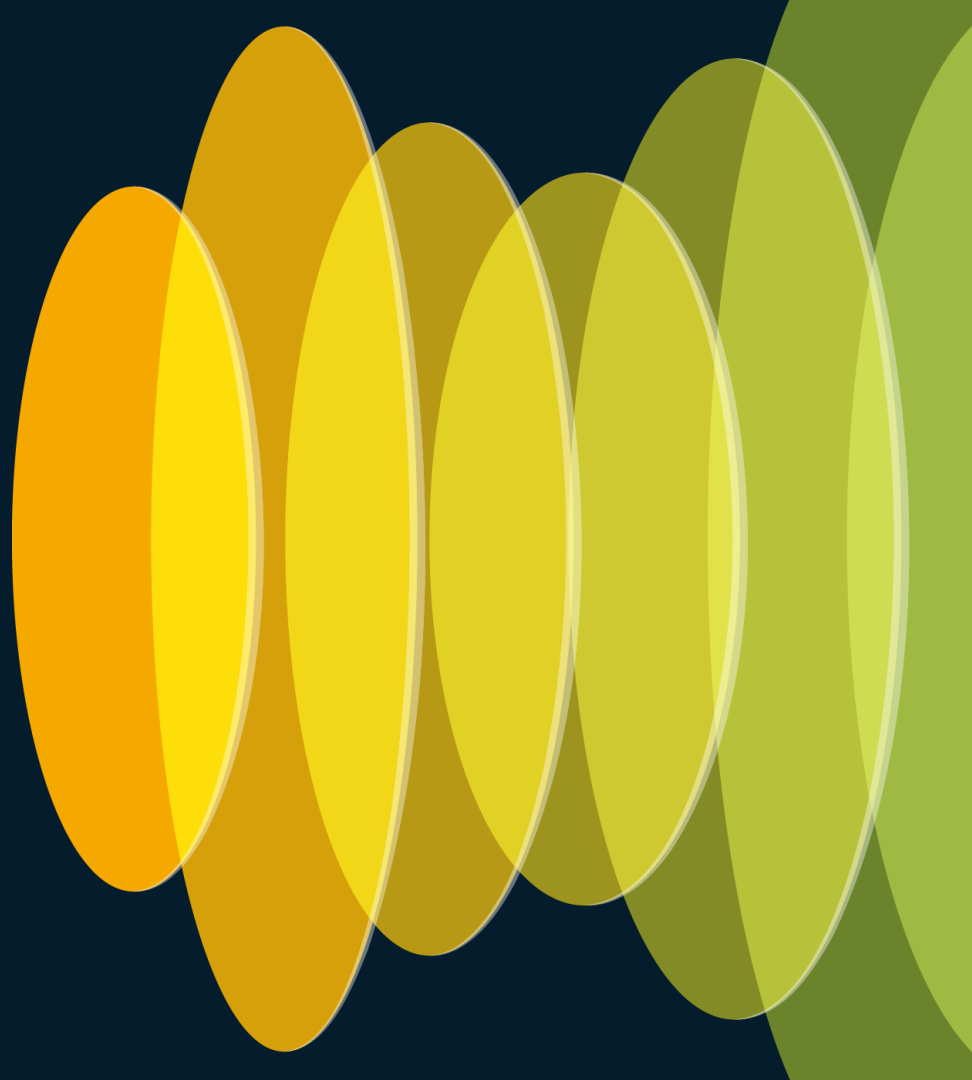
```
native/interface/GigabitEthernet[1/0/5]/shutdown
```

# Native Pull-Based Telemetry

- Pull telemetry that's cached on device
- Meraki has historically pulled telemetry via binary encoded Protobuf
- Catalyst team now serves their internal binary telemetry objects to Meraki Dashboard
- SYSLOG data will be served via the Event Logs interface



# Optimizations



# Runtime Improvements with architecture changes

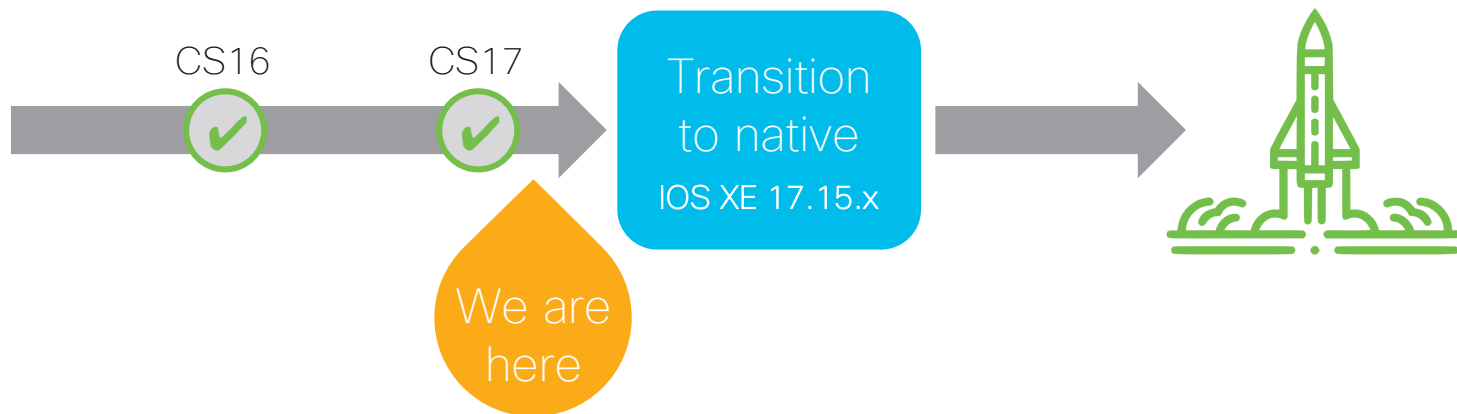
| Configuration Updates         |             |             |
|-------------------------------|-------------|-------------|
| Type                          | Version 1.0 | Version 2.0 |
| Small config update           | 30s         | 15s         |
| Full configuration deployment | 120s        | 60s         |

50% Improvement

| Boot Times    |             |             |
|---------------|-------------|-------------|
| Stack Size    | Version 1.0 | Version 2.0 |
| Single switch | 6 min       | 4 min       |
| 8M stack      | 11 min      | 4 min       |

>50% Improvement on stacks

# Seamless Upgrade Path, from 1.0 (CS) to 2.0 (XE)



All current/future installs (MS390/C9300) will have an in-path upgrade to native IOS XE management

# Onboarding for Monitoring

```
Ww_CORE_9500(config)# service meraki connect
```

Switch 1 has been successfully registered

Meraki MAC: AABBCDDEEFF

Cloud ID: QXXX-XXXX-YYYY

```
*May 7 00:16:03.519: %MERAKEI-5-SWITCH_REGISTER_SUCCESS: Switch 1 has been succesfully registered.
```

```
*May 7 00:16:03.519: %MERAKEI-5-MAC_ADDR: Meraki MAC: AABBCDDEEFF
```

```
*May 7 00:16:03.519: %MERAKEI-5-CLOUD_ID: Cloud ID: QXXX-XXXX-YYYY
```

Device Registration Status:

| Switch<br>Num | PID         | Serial<br>Number | Cloud ID       | Mac Address    | Migration<br>Status | Mode               |
|---------------|-------------|------------------|----------------|----------------|---------------------|--------------------|
| 1             | C9500-24Y4C | XXYYQCCXXC       | QXXX-XXXX-YYYY | aabb.ccdd.eeff | Registered          | C9K-C [Monitoring] |

## Monitoring App-less Onboarding

- Automated Registration and Nextunnel creation  
(`service meraki connect`)
- Simply claim the Cloud ID and add to a network

```
Service meraki connect: enable
```

```
Meraki Tunnel Config
```

```
Fetch State:
```

```
Fetch Fail:
```

```
Last Fetch(UTC):
```

```
Next Fetch(UTC):
```

```
Config Server:
```

```
Primary: usw.nt.meraki.com
```

```
Secondary: use.nt.meraki.com
```

```
Client IPv6 Addr: FD0A:9B09:1F7:1:9A18:88FF:FE00:CC00
```

```
Network Name: .Corrin - switch
```

```
Meraki Tunnel State
```

```
Primary:
```

```
Up
```

```
Secondary:
```

```
Up
```

```
Primary Last Change(UTC): 2024-05-29 23:34:02
```

```
Secondary Last Change(UTC): 2024-05-29 23:34:02
```

```
Client Last Restart(UTC): 2024-05-29 23:33:56
```

```
Meraki Tunnel Interface
```

```
Status:
```

```
Enable
```

```
Rx Packets:
```

```
795767
```

```
Tx Packets:
```

```
600246
```

```
Rx Errors:
```

```
0
```

```
Tx Errors:
```

```
0
```

```
Rx Drop Packets:
```

```
0
```

```
Tx Drop Packets:
```

```
0
```

```
Meraki Device Registration
```

```
url:
```

```
https://catalyst.meraki.com/nodes/register
```

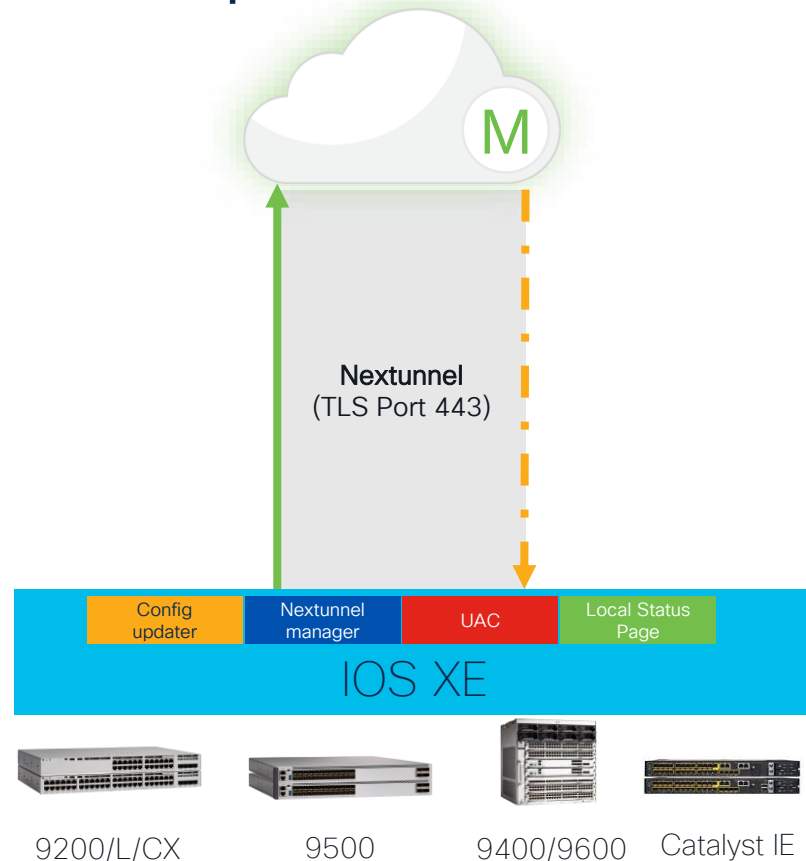
## Connection State

(`show meraki connect`)

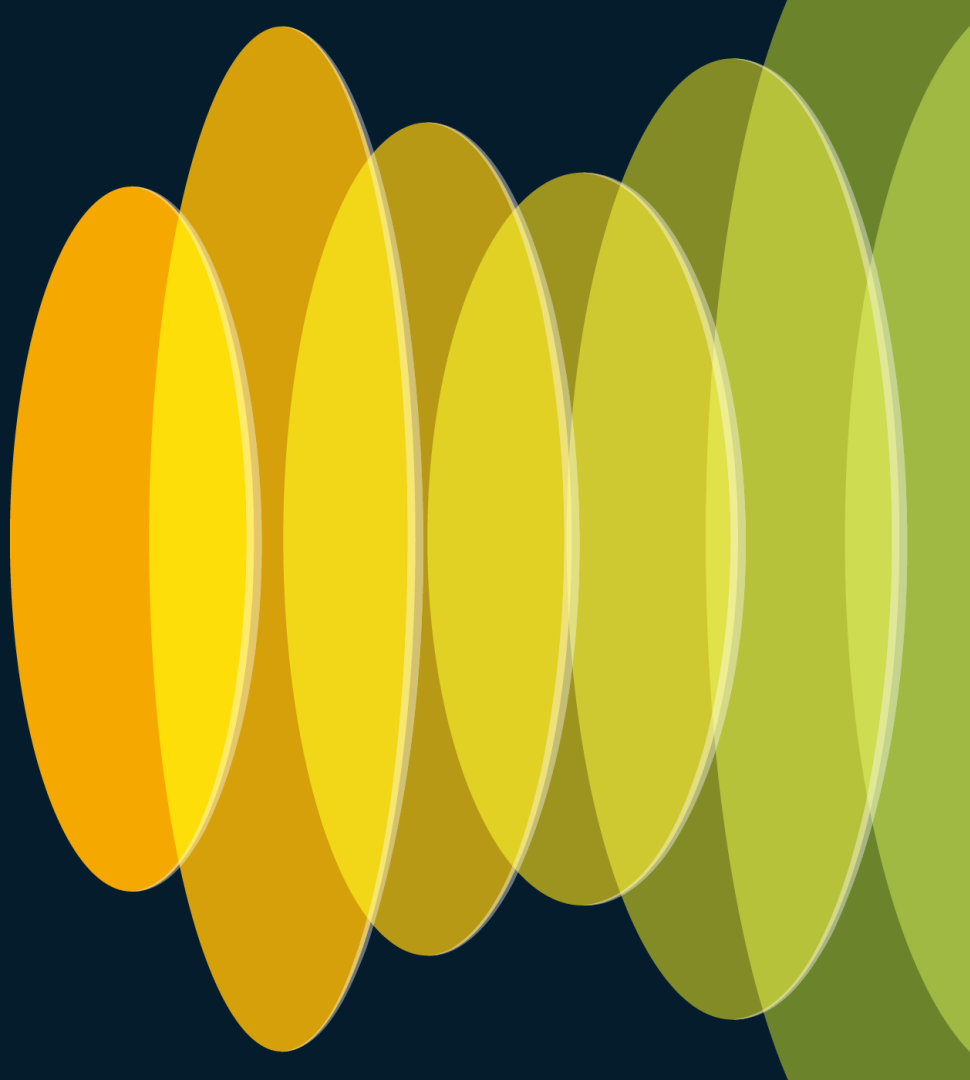
# Enabling future support for more platforms



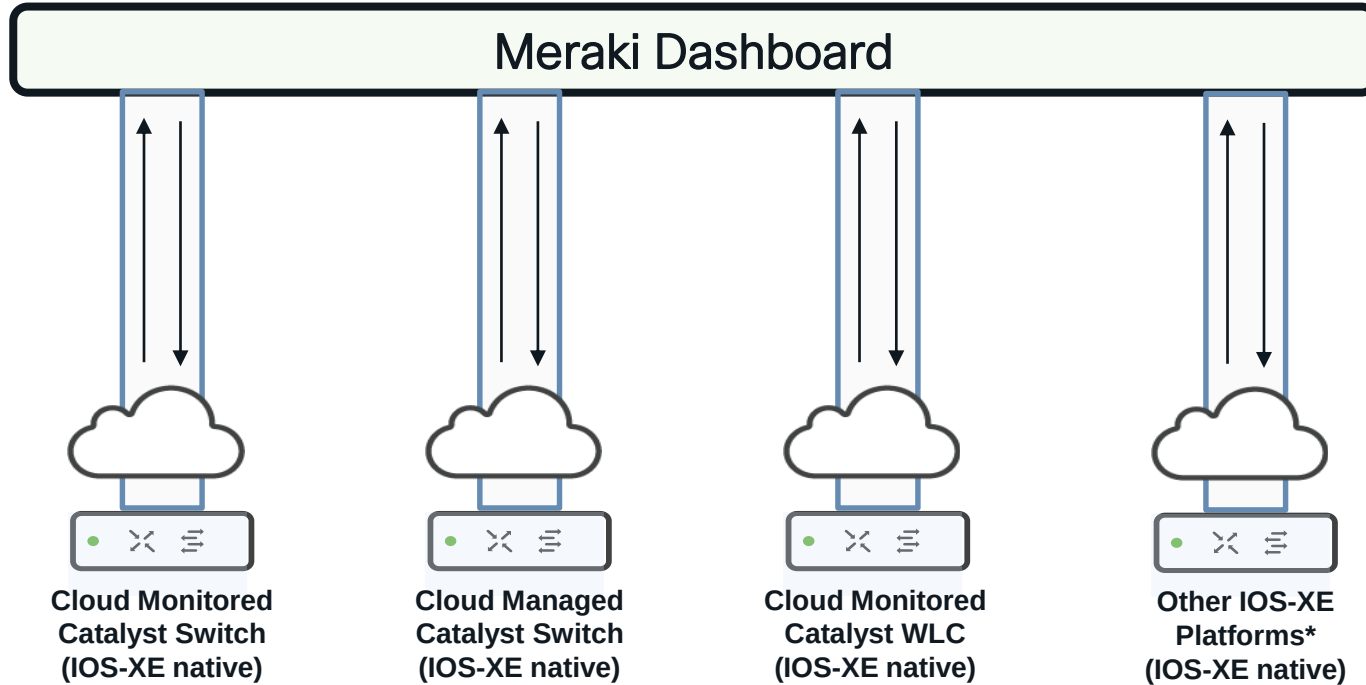
A platform independent architecture opens the door for faster support of all IOS XE switching platforms



*What does this  
unlock?*

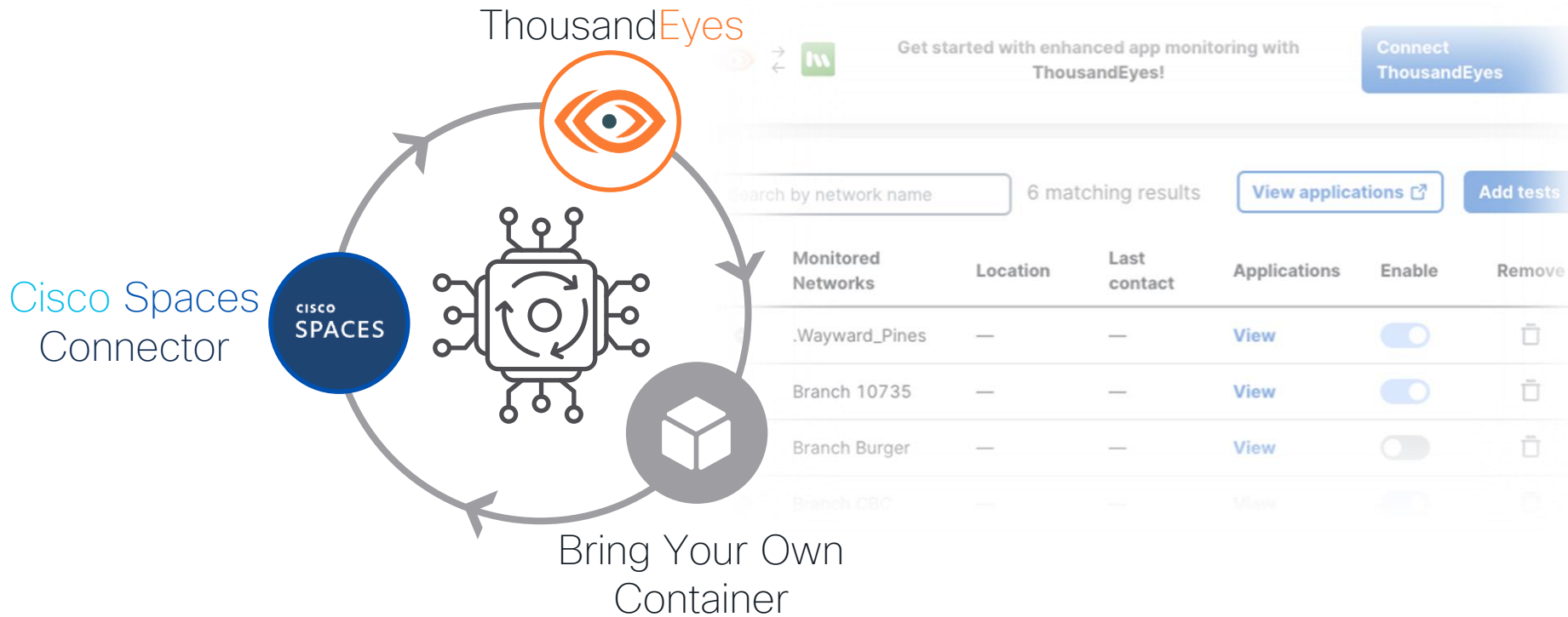


# Cloud Native IOS-XE Architecture



# Edge Compute Orchestration

*One click deployment of applications across the organization*



# Show CLI for Managed & Monitored Switches

Dashboard securely proxied CLI access

Enhancing troubleshooting with  
common CLI show commands

Ensuring accessibility to detailed  
information

## Terminal

Opens a troubleshooting console to this device, similar to an IOS-XE CLI window.

Close Terminal

Detach

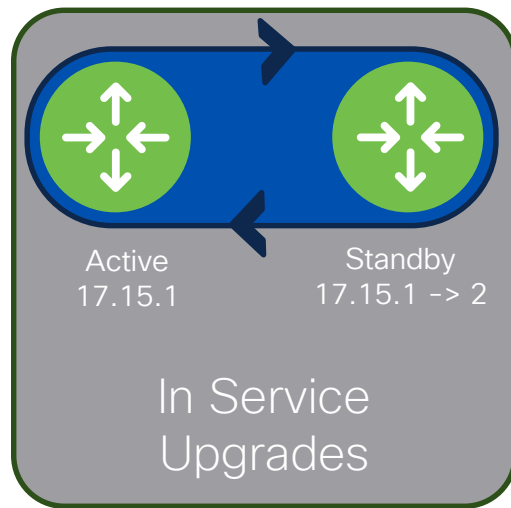
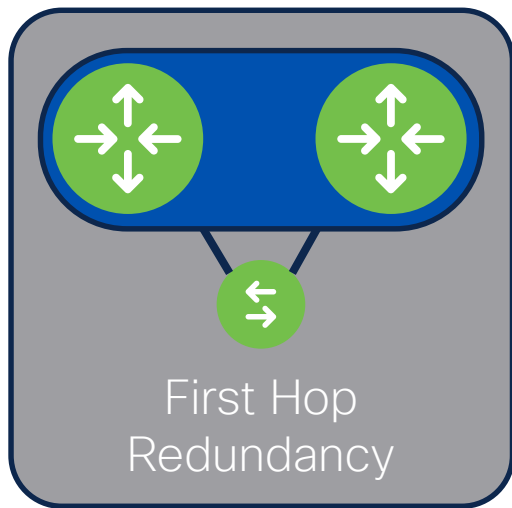
**i** This terminal allows limited access to read-only IOS-XE commands, such as **show**.

```
Ww_CORE_9300X#show ip route bgp
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
        D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
        N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
        E1 - OSPF external type 1, E2 - OSPF external type 2, m - OMP
        n - NAT, Ni - NAT inside, No - NAT outside, Nd - NAT DIA
        i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
        ia - IS-IS inter area, * - candidate default, U - per-user static route
        H - NHRP, G - NHRP registered, g - NHRP registration summary
        o - ODR, P - periodic downloaded static route, l - LISP
        a - application route
        + - replicated route, % - next hop override, p - overrides from PfR
        & - replicated local route overrides by connected

Gateway of last resort is 10.10.254.2 to network 0.0.0.0

    10.0.0.0/8 is variably subnetted, 86 subnets, 7 masks
B       10.10.0.0/16 [200/0] via 10.10.250.11, 02:09:58
B       10.12.5.0/24 [20/0] via 10.10.0.30, 08:31:41
```

# Cloud-Simple Resiliency



# Routed Interface Support



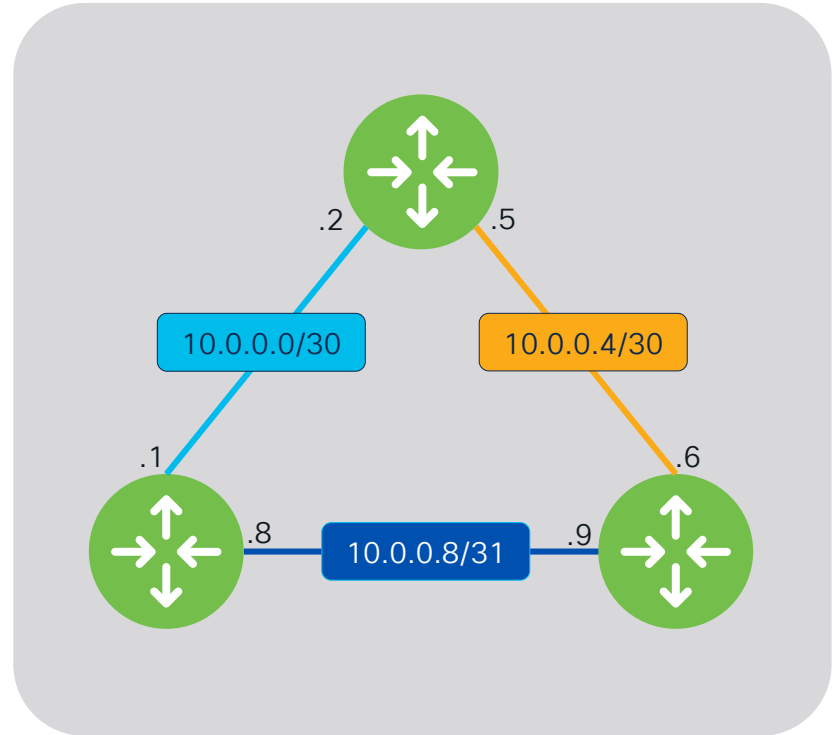
Routed mode for switchport interfaces



Reduction in STP impact areas



Enabling redundant forwarding w/o LACP



# VRF

## VRF Profiles

 Search

[+ Add VRF Profile](#)

| <input type="checkbox"/> | ID | Name      | Route Distinguisher | Route Target | Associated Networks                                                             |
|--------------------------|----|-----------|---------------------|--------------|---------------------------------------------------------------------------------|
| <input type="checkbox"/> | 1  | VRF Red   | 64513:1             | 64513:1      | HQ-Campus-Denver , Branch-Tokyo <a href="#">edit</a> <a href="#">delete</a>     |
| <input type="checkbox"/> | 1  | VRF Green | 65500:100           | 65500:100    | HQ-Campus-Denver, HQ-Campus-Phoenix <a href="#">edit</a> <a href="#">delete</a> |

 Rows per page 2 < 1 2 ... 10 >


Defined at the Organization



Implemented at the Network



Building block for complex topologies

### Add VRF Profile

Name

 WirelesslyWired\_PROD

Route Distinguisher

 65501:10

Route Target Export

 65501:10

Route Target Import

 65501:100

[+ Add Route-Target](#)
[Cancel](#)
[Save](#)

# Border Gateway Protocol (BGP)



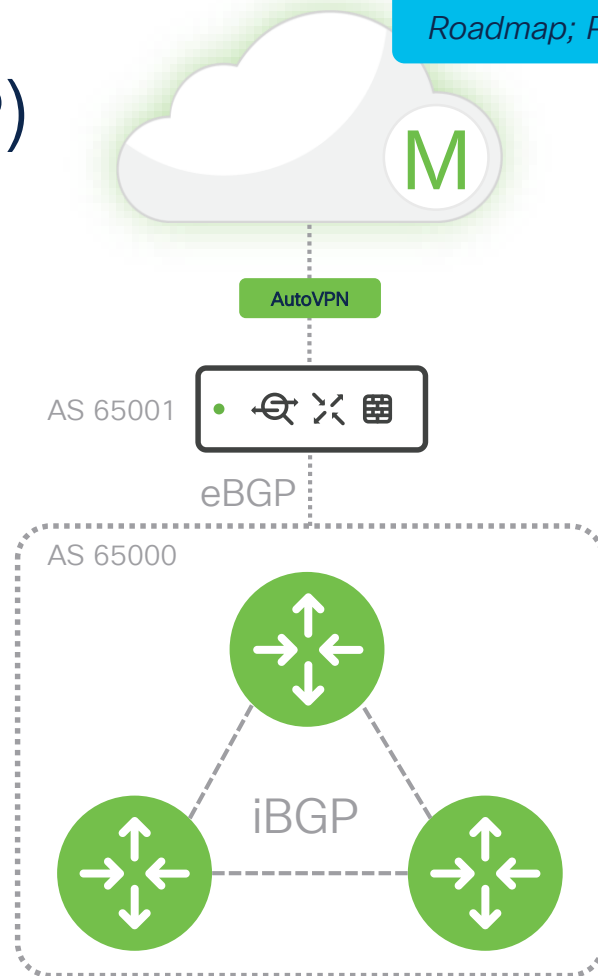
iBGP/eBGP support on 390/9300/9500



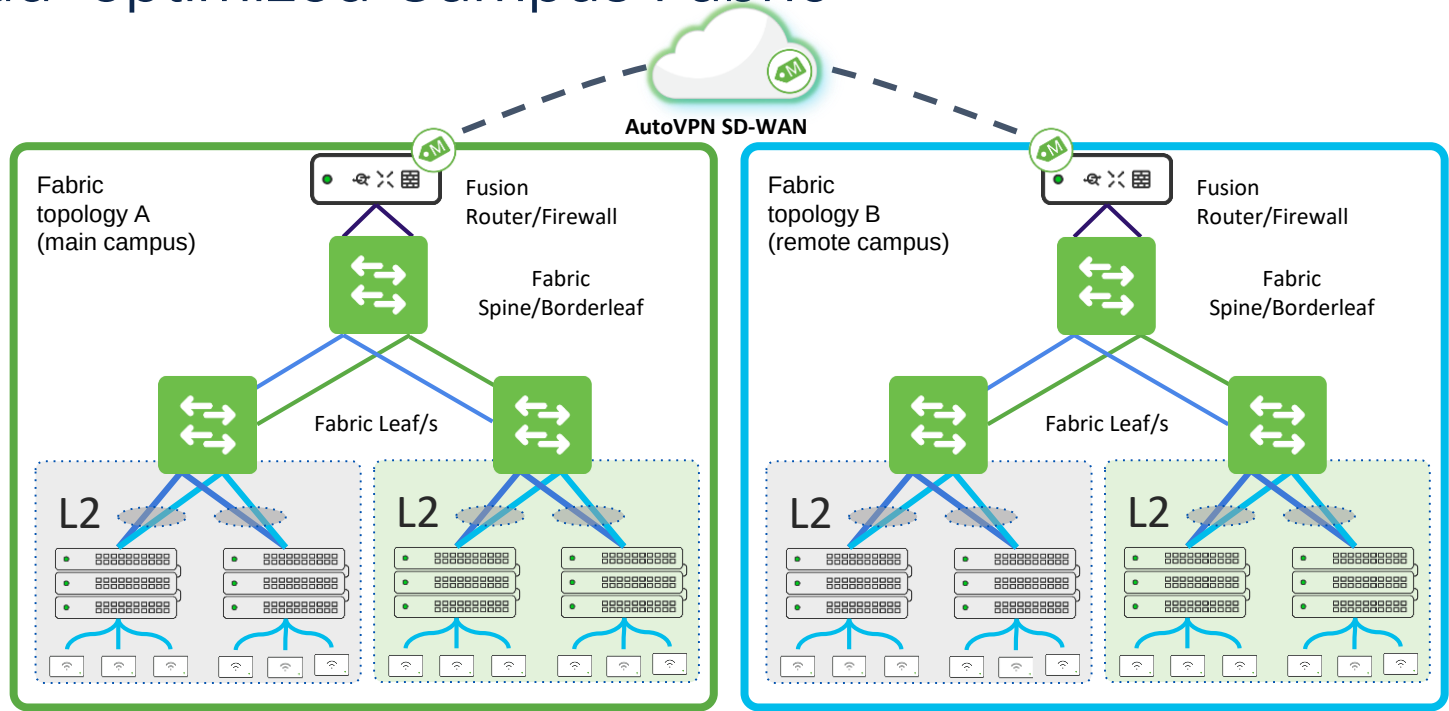
MP-BGP with support for VRFs



Full interoperability with MX  
and standard BGP capable platforms



# Cloud-optimized Campus Fabric



Focus on  
brownfield with  
seamless  
migration

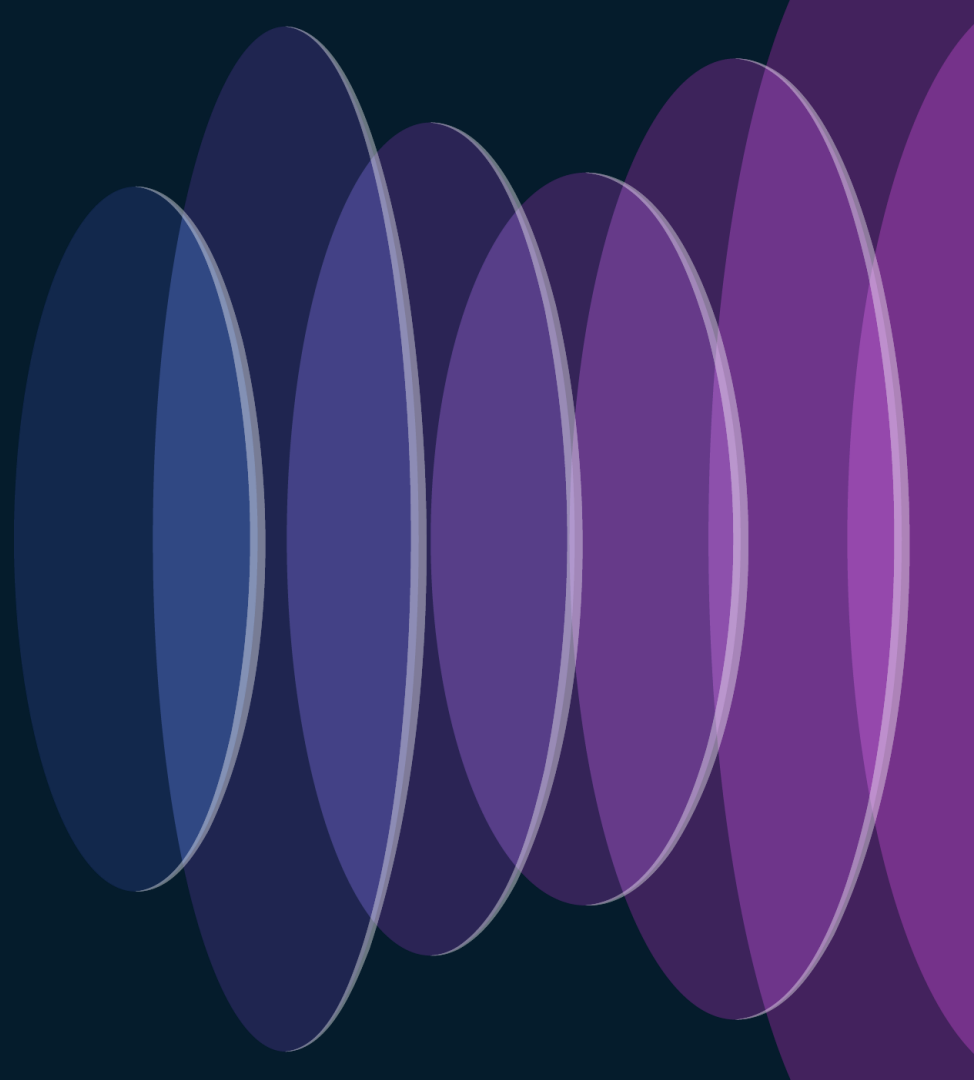
Standards-based  
BGP-EVPN Fabric

Optimized L2  
mobility through  
Anycast Gateways

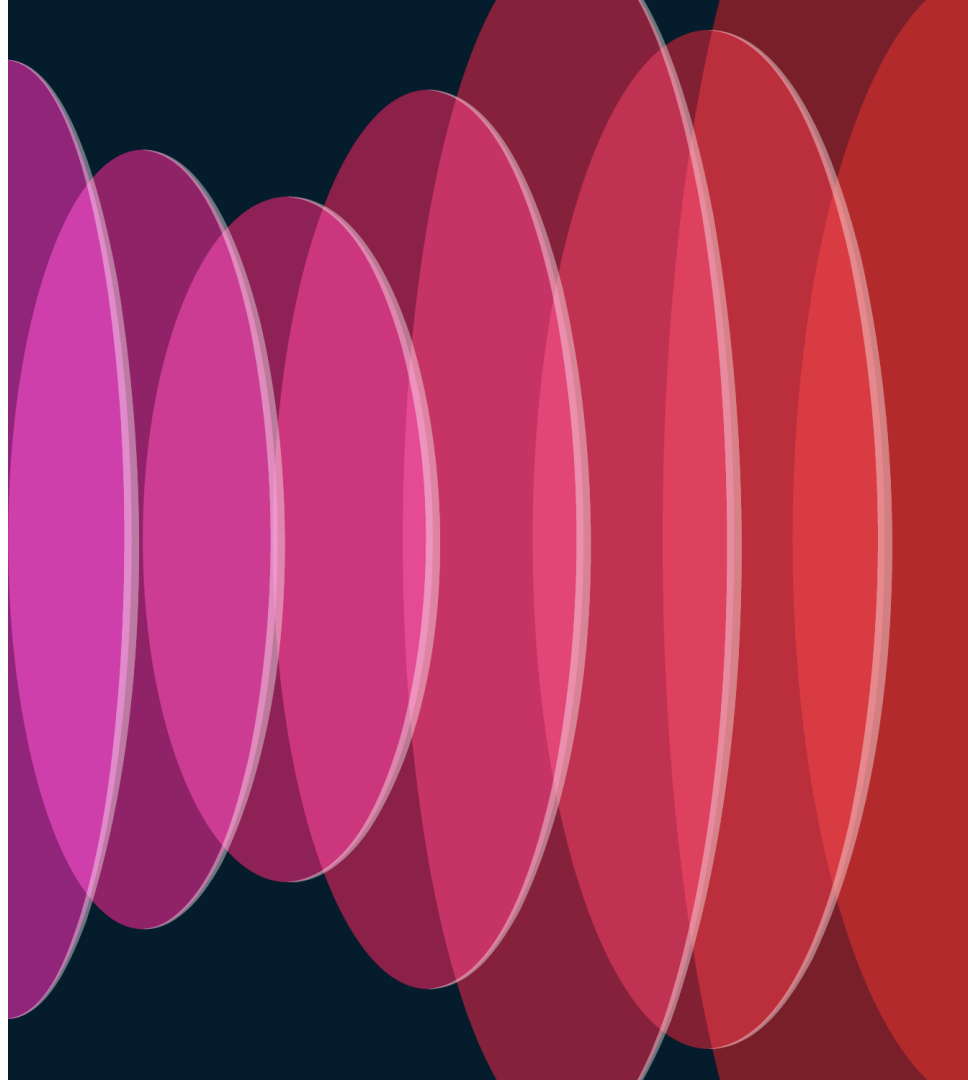
Adaptive Policy  
For micro-  
segmentation and  
ZTNA

Leverage Existing  
Hardware Where  
Possible  
(390/9300/9500)

# Conclusion

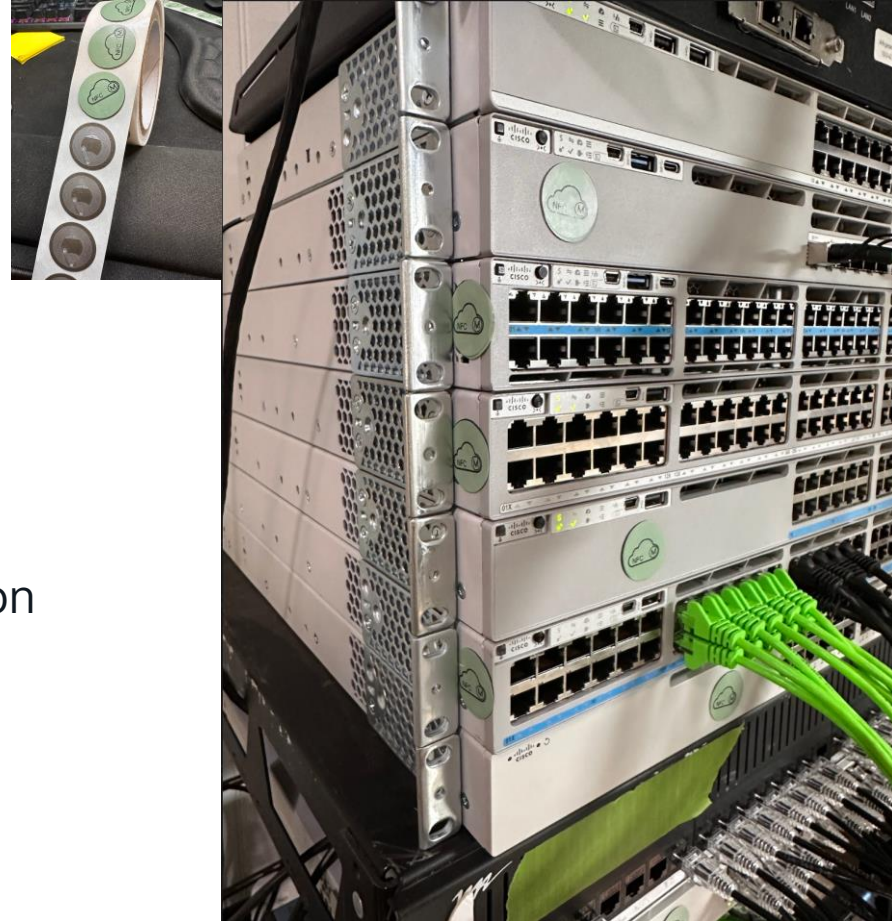


Now for a little fun...



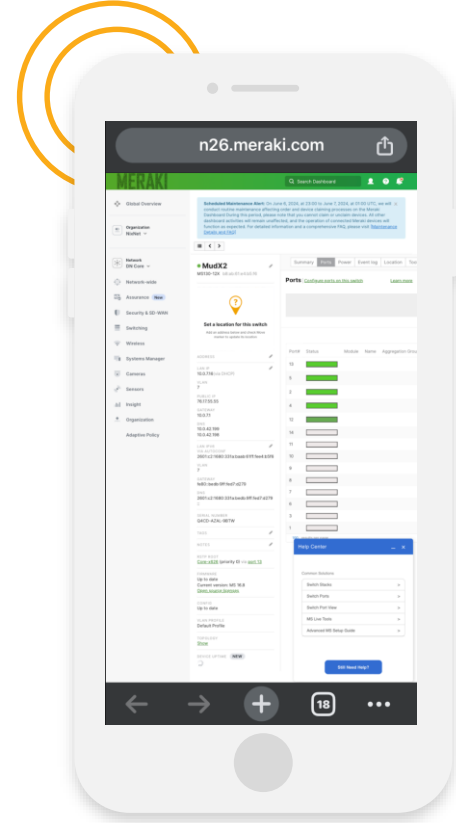
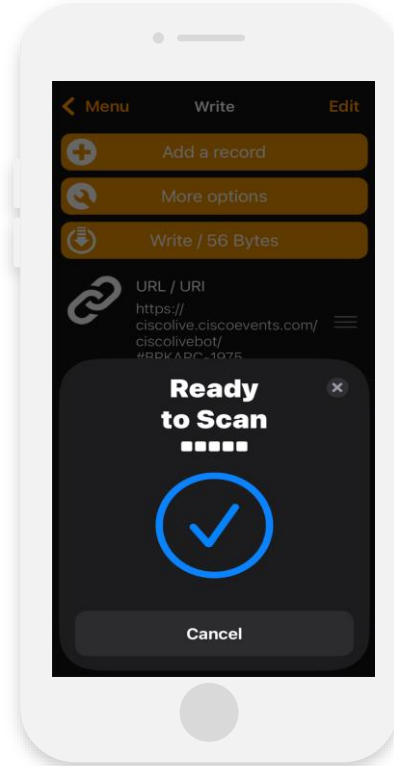
# Sticker Time!

- Re-programmable NFC Sticker
  - NXP-NTAG215 chip 540 bytes
  - Metal friendly backing
  - Password security
- Great for
  - Dashboard Link to Switch configuration
  - Social Media “Tap to connect”
  - Apple/Android Shortcut Automations
  - Connect to WiFi Network



# Sticker Time!

1. Download “NFC Tools”
  - Apple or Android store
2. Write TAG
  - Add Record (URI/URL)
  - Add Social Media
3. Read by Tapping with phone
  - Apple reader located upper left
  - Android reader dead center



# Complete Your Session Evaluations



Complete a minimum of 4 session surveys and the Overall Event Survey to be entered in a drawing to **win 1 of 5 full conference passes** to Cisco Live 2025.

---



**Earn 100 points** per survey completed and compete on the Cisco Live Challenge leaderboard.

---



Level up and earn **exclusive prizes!**

---



Complete your surveys in the **Cisco Live mobile app**.

# Continue your education

- Visit the Cisco Showcase for related demos
- Book your one-on-one Meet the Engineer meeting
- Attend the interactive education with DevNet, Capture the Flag, and Walk-in Labs
- Visit the On-Demand Library for more sessions at [www.CiscoLive.com/on-demand](https://www.CiscoLive.com/on-demand)

Contact me at: [ndarrow@cisco.com](mailto:ndarrow@cisco.com)



The bridge to possible

# Thank you

CISCO *Live!*

#CiscoLive